

Natural Language Processing (NLP) Lab: Text Preprocessing and Spelling Correction V2

Welcome to the comprehensive lab guide for Natural Language Processing. In this module, we will explore the foundational techniques of text preprocessing and implement an edit distance algorithm for spelling correction. Raw text data is inherently noisy, unstructured, and filled with grammatical inconsistencies. Before feeding text into machine learning models, it is imperative to clean, normalize, and structure it.

1. Text Cleaning with Regular Expressions (Regex)

Regular expressions provide a versatile language for matching character patterns within text. In NLP, regex is typically the first step in the pipeline. Web-scraped text often contains HTML tags, URLs, metadata, and special characters that carry no semantic value. By applying regex, we can systematically strip these out, leaving only the linguistic content.

Core Rules and Syntax of Regular Expressions

Regex operations rely on a set of standard structural rules, characters, and quantifiers to identify patterns:

Category	Pattern	Description & Meaning
Metacharacters	.	Matches any single character except a newline.
	^	Matches the start of the string.
	\$	Matches the end of the string.

Category	Pattern	Description & Meaning
Quantifiers	*	Matches 0 or more repetitions of the preceding pattern.
	+	Matches 1 or more repetitions of the preceding pattern.
	?	Matches 0 or 1 repetition (makes the pattern optional).
	{n,m}	Matches between n and m repetitions.
Character Classes	[A-Za-z]	Matches any uppercase or lowercase letter.
	\d	Matches any decimal digit (equivalent to [0-9]).
	\s	Matches any whitespace character (space, tab, newline).
	[^...]	Inverted character class; matches any character NOT in the brackets.

```
import re
```

```

# Sample messy text
raw_text = "Hello!!! Welcome to the NLP lab in 2026. This text has <br> HTML tags,
numbers like 123, and symbols #$$%^."

# 1. Remove HTML tags
no_html = re.sub(r'<[^>]+>', "", raw_text)

# 2. Remove special characters and numbers (keep only letters and spaces)
cleaned_text = re.sub(r'[^A-Za-z\s]', "", no_html)

# 3. Remove extra whitespaces
cleaned_text = re.sub(r'\s+', ' ', cleaned_text).strip()

print("Cleaned Text :", cleaned_text)

```

2. Tokenization and Stop Word Removal

Tokenization is the algorithmic process of chopping a character sequence into distinct pieces, known as tokens. While word tokenization is most common, sentence tokenization is also used. Tokenization relies heavily on language heuristics (e.g., handling punctuation and contractions correctly).

Stop Words are the most commonly occurring words in a language (e.g., "the", "is", "at", "which"). Because they appear in almost every document, they provide little to no signal for distinguishing between different texts in tasks like classification. Removing them drastically reduces the vocabulary size and computational complexity.

```

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk

nltk.download('punkt')
nltk.download('stopwords')

tokens = word_tokenize(cleaned_text.lower())
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word not in stop_words]

```

```
print("Tokens after stop word removal:", filtered_tokens)
```

3. Word Normalization: Stemming vs. Lemmatization

Language is highly inflected. The words "run", "runs", "running", and "ran" all refer to the same underlying concept. Normalization aims to reduce inflectional forms to a common base form.

Porter's Stemming Algorithm Rules

Stemming is a crude, rule-based heuristic that chops off word endings. The **Porter Stemmer** is defined by a series of 5 sequential phases containing algorithmic condition checks based on the measure m of a word (the count of vowel-consonant sequences):

- **Step 1a:** Handles basic plurals and standard terminal configurations.
 - $sses \rightarrow ss$ (e.g., *caresses* $\rightarrow$ *caress*)
 - $ies \rightarrow i$ (e.g., *ponies* $\rightarrow$ *poni*)
 - $ss \rightarrow ss$ (e.g., *tie* $\rightarrow$ *tie*)
 - $s \rightarrow \emptyset$ (e.g., *cats* $\rightarrow$ *cat*)
- **Step 1b:** Targets inflected verbs ending in eed, ed, or ing.
 - $eed \rightarrow ee$ if the preceding root has $m > 0$ (e.g., *agreed* $\rightarrow$ *agree*)
 - ed / ing are stripped if there is a vowel in the preceding stem (e.g., *plastered* $\rightarrow$ *plaster*, *walking* $\rightarrow$ *walk*)
- **Step 1c:** Turns a terminal y to i if preceded by another letter (e.g., *happy* $\rightarrow$ *happi*).
- **Step 2:** Maps compound suffixes down to simpler variants for roots where $m > 0$.
 - $ational \rightarrow ate$ (e.g., *relational* $\rightarrow$ *relate*)
 - $izer \rightarrow ize$ (e.g., *digitizer* $\rightarrow$ *digitize*)
 - $alli \rightarrow al$ (e.g., *radically* $\rightarrow$ *radical*)
- **Step 3:** Handles specialized derivational suffixes for roots where $m > 0$.
 - $icate \rightarrow ic$ (e.g., *triplicate* $\rightarrow$ *triplic*)
 - $ative \rightarrow \emptyset$ (e.g., *formative* $\rightarrow$ *form*)
 - $alize \rightarrow al$ (e.g., *formalize* $\rightarrow$ *formal*)
- **Step 4:** Strips absolute derivational standard endings if $m > 1$.

- Removes suffixes like -al, -ance, -ence, -er, -ic, -able, -ible (e.g., *allowance* → *allow*, *effective* → *effect*).
- **Step 5:** Cleans up terminal vowels and double consonants.
 - **Step 5a:** Removes trailing e if m > 1 (e.g., *probate* → *probat*).
 - **Step 5b:** Changes terminal double consonants to single consonants if m > 1 and ends in ll (e.g., *control* → *control*).

Lemmatization

Lemmatization performs a full morphological analysis. It requires a detailed dictionary (like WordNet) and knowledge of the word's Part-of-Speech (POS) to return the correct dictionary base form (the *lemma*). For example, it knows that the base form of "better" (adjective) is "good".

Feature	Stemming	Lemmatization
Approach	Rule-based suffix stripping	Dictionary lookup and morphological analysis
Speed	Fast and computationally inexpensive	Slower due to vocabulary mapping
Output	Often non-real words (Stems)	Always actual dictionary words (Lemmas)

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
import nltk
nltk.download('wordnet')

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

word = "running"
print("Stemmed:", stemmer.stem(word))
print("Lemmatized (verb):", lemmatizer.lemmatize(word, pos='v'))
```

4. Edit Distance (Levenshtein) and Spelling Correction

The **Levenshtein Edit Distance** algorithm mathematically quantifies how dissimilar two strings are. It computes the minimum number of character-level operations required to transform one string into another.

Mathematical Recurrence Formula

Given two strings A of length m and B of length n , the distance $D(i, j)$ between their prefixes $A[1..i]$ and $B[1..j]$ is defined recursively as follows:

```

    | max(i, j)                                if min(i, j) == 0
D(i, j) = |
    | min( D(i-1, j) + 1,                      # Deletion of A[i]
    |      D(i, j-1) + 1,                      # Insertion of B[j]
    |      D(i-1, j-1) + cost )                # Substitution
```

Where:

cost = 0 if $A[i] == B[j]$

cost = 1 if $A[i] != B[j]$

To calculate this efficiently, we use **Dynamic Programming**. We construct an $m \times n$ matrix. Each cell (i, j) represents the minimum edit distance up to those indices.

```
import numpy as np

def calculate_edit_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = np.zeros((m + 1, n + 1), dtype=int)

    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                cost = 0
```

```

        else:
            cost = 1

            dp[i][j] = min(
                dp[i - 1][j] + 1,    # Deletion
                dp[i][j - 1] + 1,    # Insertion
                dp[i - 1][j - 1] + cost # Substitution
            )
        return dp[m][n]

# Spelling Correction System
vocabulary = ["apple", "banana", "orange", "grape", "mango"]
misspelled_word = "bananna"

distances = {valid_word: calculate_edit_distance(misspelled_word, valid_word) for
valid_word in vocabulary}
sorted_distances = sorted(distances.items(), key=lambda item: item[1])

print(f"Did you mean: '{sorted_distances[0][0]}' instead of '{misspelled_word}'?")

```

Natural Language Processing (NLP) Lab 2: Text Representation, N-Gram Language Modeling, and Shannon's Guessing Game

Welcome to the comprehensive guide for NLP Lab 2. Having explored cleaning and normalisation in Lab 1, this lab focuses on text representation (converting words to vectors) and probabilistic language modeling. These concepts underwrite key technologies from search engine algorithms to generative large language models.

1. Mathematical Formulations of Vector Space Models

Before computers can apply algebraic algorithms to text, the unstructured natural language must be translated into structured vector formats. We explore the two core methods of vector space representation.

1.1 Bag of Words (BoW) Representation

The Bag of Words model simplifies documents by stripping away syntax, grammatical rules, and word order, preserving only the frequency of terms. Let $D = \{d_1, d_2, \dots, d_M\}$ be a corpus containing M documents, and let $V = \{v_1, v_2, \dots, v_N\}$ be the vocabulary list of N unique tokens extracted from the corpus D .

The vector representation $\mathbf{x}_d$ of a document $d \in D$ is defined as:

$$\mathbf{x}_d = [f(v_1, d), f(v_2, d), \dots, f(v_N, d)]^T$$

Where $f(v_j, d)$ represents the raw frequency (count) of the vocabulary word v_j in document d . While intuitive, BoW treats every word with equal importance, which leads to common words (like "is" or "the") dominating the vectors.

1.2 Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF resolves the limitation of BoW by weighting terms based on how unique they are to a specific document. The mathematical formulation combines two distinct metrics:

Term Frequency (TF): Measures the density of a term t in document d . While raw frequency can be used, standard normalization adjusts for document length:

$$TF(t, d) = f(t, d) / \sum_{t' \in d} f(t', d)$$

Inverse Document Frequency (IDF): Measures how much information a word provides across the entire corpus D . Terms that appear everywhere have a low IDF, while rare terms have a high IDF:

$$IDF(t, D) = \log(|D| / (1 + |\{ d \in D : t \in d \}|))$$

Note: Adding 1 to the denominator prevents division-by-zero errors in case the term does not exist in any document. In libraries like Scikit-Learn, smoothing is applied using the formula:

$$IDF(t) = \log((1 + |D|) / (1 + DF(t))) + 1.$$

The Joint TF-IDF Score: The final value is the product of TF and IDF:

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

2. Mathematical Formulations of N-Gram Language Modeling

An N-gram model estimates the probability of a sequence of words occurring. By the **Chain Rule of Probability**, the joint probability of a sequence of words $W = (w_1, w_2, \dots, w_k)$ is computed as:

$$P(w_1, w_2, \dots, w_k) = \prod_{i=1..k} P(w_i | w_1, w_2, \dots, w_{i-1})$$

Tracking a long history of preceding words is computationally infeasible. To solve this, we apply the **Markov Assumption**, which assumes the probability of a word depends only on a limited window of $N-1$ preceding words:

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-N+1}, \dots, w_{i-1})$$

For a **Bigram Model (N=2)**, the current word depends only on the immediate preceding word:

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-1})$$

For a **Trigram Model (N=3)**, the current word depends on the two preceding words:

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-2}, w_{i-1})$$

2.1 Maximum Likelihood Estimation (MLE)

To train the model, we calculate these conditional probabilities from a training corpus using Maximum Likelihood Estimation (MLE). This involves counting the occurrences of N-gram sequences:

$$P_{MLE}(w_i | w_{i-N+1}, \dots, w_{i-1}) = C(w_{i-N+1}, \dots, w_{i-1}, w_i) / C(w_{i-N+1}, \dots, w_{i-1})$$

Where $C(\bullet)$ represents the count of a specific sequence in the corpus. For example, in a Bigram model, the probability of the word "processing" given "language" is: $P(\text{processing} | \text{language}) = C(\text{language processing}) / C(\text{language})$.

2.2 Concrete Step-by-Step N-Gram Calculation Example

To understand how counts translate into Maximum Likelihood Estimation (MLE) probabilities, let us construct a manual calculation using a toy corpus. We will use sentence boundary markers (<s> for sentence start and </s> for sentence end) to handle edge cases cleanly.

The Toy Corpus

- **Sentence 1:** <s> I am Sam </s>
- **Sentence 2:** <s> Sam I am </s>
- **Sentence 3:** <s> I am free </s>

Step 1: Unigram Frequency Baseline (Denominators)

Before computing conditional probabilities, we calculate the raw frequency of each unique token. These counts serve as the history denominators in our probability calculations.

Token (w)	Raw Count C(w)

<s>	3
I	3
am	3
Sam	2
free	1
</s>	3

Step 2: Bigram Probability Table Calculation

For a Bigram model (N=2), the probability of a word depends solely on its single immediate predecessor. We slide a window of size 2 across our corpus and apply the bigram MLE formula:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

History (w_{i-1})	Target Word (w_i)	Bigram Count $C(w_{i-1}, w_i)$	History Count $C(w_{i-1})$	MLE Probability $P(w_i w_{i-1})$
<s>	I	2	3	$2/3 \approx 0.67$
<s>	Sam	1	3	$1/3 \approx 0.33$

I	am	3	3	$3/3 = 1.00$
am	Sam	1	3	$1/3 \approx 0.33$
am	free	1	3	$1/3 \approx 0.33$
am	</s>	1	3	$1/3 \approx 0.33$
Sam	</s>	1	2	$1/2 = 0.50$
Sam	I	1	2	$1/2 = 0.50$
free	</s>	1	1	$1/1 = 1.00$

Observation: Notice how the probabilities starting from any given history (like **am** transitioning to **Sam**, **free**, or **</s>**) naturally sum up to **1.00** ($0.33 + 0.33 + 0.33 \approx 1.00$).

Step 3: Trigram Probability Table Calculation

For a Trigram model ($N=3$), the context window expands. The probability of a word depends on the **two** preceding words. We slide a window of size 3 across the corpus and apply the trigram MLE formula:

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

History (w_{i-2}, w_{i-1})	Target Word (w_i)	Trigram Count $C(w_{i-2}, w_{i-1}, w_i)$	Bigram History Count $C(w_{i-2}, w_{i-1})$	MLE Probability $P(w_i \mid w_{i-2}, w_{i-1})$

		)		
<s>, I	am	2	2	$2/2 = 1.00$
<s>, Sam	I	1	1	$1/1 = 1.00$
I, am	Sam	1	3	$1/3 \approx 0.33$
I, am	free	1	3	$1/3 \approx 0.33$
I, am	</s>	1	3	$1/3 \approx 0.33$
am, Sam	</s>	1	1	$1/1 = 1.00$
Sam, I	am	1	1	$1/1 = 1.00$
am, free	</s>	1	1	$1/1 = 1.00$

Key takeaway: The trigram history context is more specific. When the history is I, am, the model distributes probability equally (0.33 each) among three possible sequels: Sam, free, and </s>. However, for highly unique histories like am, free, the transition is completely deterministic (1.00) because there is only one observed sequence.

3. Detailed Function Documentation

The following table outlines the programmatic architecture of our lab code, mapping the functions used to their inputs, outputs, and design mechanics:

Function Name	Inputs	Outputs	Functional Description & Logic
full_preprocess	- text (str) - remove_stopwords (bool) - use_lemmatization (bool)	tokens (list of str)	Cleans and tokenizes raw text. Converts characters to lowercase, strips HTML elements with regex, removes non-alphabetic symbols, splits sentences into discrete words, conditionally filters out English stop words, and applies NLTK's WordNetLemmatizer to resolve morphological inflections.
build_ngram_model	- tokens (list of str) - n (int)	prob_model (defaultdict of dicts)	Constructs an N-gram frequency distribution. It iterates over the token list with a sliding window of size n, mapping a history tuple of length n-1 to a Counter tracking succeeding words.

Function Name	Inputs	Outputs	Functional Description & Logic
			Finally, it divides the absolute count of each target word by the cumulative occurrence of its history, outputting normalized MLE probabilities.
shannons_predictor	- history_phrase (str) - model (dict) - n (int)	sorted_predictions (list of tuples)	Simulates Shannon's next-word guessing experiment. Preprocesses the user's prompt (leaving stop words to maintain phrase syntax), extracts the exact trailing sequence of size n-1 as a key, checks the key against the trained N-gram model, and returns matches sorted by confidence probability in descending order.
generate_text	- model (dict) - seed_phrase (str) - max_words (int)	generated_sequence (str)	Generates text sequences. It initializes a local sequence list with

Function Name	Inputs	Outputs	Functional Description & Logic
	- n (int)		preprocessed seed words, loops up to max_words, extracts the current history, selects the next word using np.random.choice weighted by the model's probabilities, and appends the selection to generate continuous text.

4. In-Depth Function Execution Mechanics

4.1 full_preprocess

This function acts as our primary cleaning pipeline, processing text through the following stages:

1. **Case Folding:** Standardizes text by converting characters to lowercase using `text.lower()`.
2. **Regex Filtering:** Applies `re.sub(r'<[^>]+>', "", text)` to eliminate HTML tags and `re.sub(r'^[a-z\s]', "", text)` to remove numbers, punctuation, and special characters.
3. **Tokenization:** Uses `word_tokenize(text)` to split sentences into individual tokens.
4. **Stop Word Removal:** If `remove_stopwords=True`, filters out standard high-frequency English functional words (e.g., "and", "the").
5. **Lemmatization:** If `use_lemmatization=True`, applies `WordNetLemmatizer().lemmatize()` to map morphological variations back to their base dictionary forms (lemmas).

4.2 build_ngram_model

This function acts as our model compiler, building probability lookup structures:

1. **Token Loop & History Parsing:** Iterates through the list of input tokens. For each index i , it captures a history key as a tuple: `tokens[:i+n-1]`. The corresponding next word is captured at index $i+n-1$.
2. **Count Accumulation:** Uses a nested structure `defaultdict(Counter)` to increment the frequency of the next word given its history: `model[history][next_word] += 1`.
3. **MLE Normalization:** Iterates through the compiled history keys. For each key, it sums up the total counts of all seen target words ($\sum C(history, w)$). It then divides each individual target word count by this total sum to calculate and store the normalized MLE probability.

4.3 shannons_predictor

This function replicates Claude Shannon's 1951 experiment, testing how well a model can predict the next word in a sequence:

1. **Context Preprocessing:** Runs the prompt through `full_preprocess`, keeping stop words (`remove_stopwords=False`) to preserve syntactic structure.
2. **History Extraction:** Extracts the final $n-1$ tokens of the processed input as the history key: `tuple(processed_history[-(n-1):])`.
3. **Probability Query & Sort:** If the history key exists in the model, it retrieves the next-word candidates and their probabilities. It then sorts these candidates in descending order based on their probability values using `sorted(..., key=lambda x: x[1], reverse=True)`.

4.4 generate_text

This function acts as a generative engine, synthesizing new sentences based on the trained model:

1. **Sequence Initialization:** Preprocesses the `seed_phrase` to establish the starting context for generation.
2. **Generative Loop:** Runs a loop for a maximum of `max_words`. In each iteration:
 - It extracts the active history key: `tuple(output_tokens[-(n-1):])`.
 - If the history key exists in the model, it splits candidates and their respective probabilities into two parallel lists: `choices` and `probabilities`.
 - It uses `np.random.choice(choices, p=probabilities)` to select the next word. This ensures that words with higher probabilities are more likely to be chosen while still maintaining natural variation.
 - The chosen word is appended to `output_tokens`, updating the history key for the next iteration.
3. **Output Formatting:** Joins the final sequence list into a single string space-separated output block using `" ".join()`.

5. Reference Sheet for Python & Library Built-in Functions

Below is a reference guide documenting the built-in Python and library functions utilized in the code:

5.1 Python Standard Library Built-ins

`re.sub(pattern, repl, string)`

- **Purpose:** Searches a string for a regular expression pattern and replaces all matches with a replacement string.
- **Inputs:** `pattern` (str/regex), `repl` (str replacement), `string` (source text).
- **Role in Lab:** Strips HTML tags and removes non-alphabetic characters during text cleaning.

`sorted(iterable, key=None, reverse=False)`

- **Purpose:** Returns a new sorted list from the items in any iterable.
- **Inputs:** `iterable` (collection), `key` (sorting evaluation function), `reverse` (bool).
- **Role in Lab:** Sorts next-word candidates in Shannon's game by probability in descending order.

`tuple(iterable)`

- **Purpose:** Converts an iterable collection (like a list) into an immutable, hashable tuple.
- **Role in Lab:** Converts N-gram history lists (e.g., ['i', 'love']) into tuples so they can be used as dictionary keys.

`sum(iterable)`

- **Purpose:** Computes the sum of all numerical values inside an iterable.
- **Role in Lab:** Sums raw counts of all next-word candidates to calculate denominators for MLE normalization.

5.2 collections Container Module

`defaultdict(default_factory)`

- **Purpose:** A dictionary subclass that automatically initializes a default value when a missing key is accessed.
- **Role in Lab:** Simplifies nested dictionary construction. If an N-gram history key is encountered for the first time, it automatically initializes a new `Counter` object.

`Counter([iterable])`

- **Purpose:** A dictionary subclass designed to count the frequencies of hashable elements in an iterable.
- **Role in Lab:** Tracks absolute frequency counts of words trailing any given N-gram history context.

5.3 NLTK (Natural Language Toolkit)

`word_tokenize(text)`

- **Purpose:** Splits an input text string into individual word and punctuation tokens.
- **Role in Lab:** Isolates words to prepare sentences for text normalization and N-gram tracking.

`stopwords.words('english')`

- **Purpose:** Retrieves a pre-compiled list of common English functional words (e.g., "the", "a", "is").
- **Role in Lab:** Provides the filtering vocabulary used to extract informative keyword words.

`WordNetLemmatizer().lemmatize(word, pos='n')`

- **Purpose:** Resolves a word to its base dictionary form (lemma) using the WordNet lexical database.
- **Role in Lab:** Normalizes morphologically varied words (e.g., "running", "runs", "ran") to their root form ("run") to keep vocabulary compact and accurate.

5.4 Scientific & Machine Learning Utilities

`np.random.choice(a, size=None, replace=True, p=None)`

- **Purpose:** Generates a random sample from a 1D array or list of options.
- **Inputs:** `a` (list of choices), `p` (associated probability distribution where $\sum p = 1.0$).
- **Role in Lab:** Used in text generation to select the next word based on calculated N-gram probabilities, mimicking natural language distributions.

`CountVectorizer()`

- **Purpose:** Converts a collection of raw text documents into a matrix of token counts (Bag of Words model).
- **Role in Lab:** Easily builds Bag of Words vector representations from a clean corpus.

`TfidfVectorizer()`

- **Purpose:** Converts a collection of raw text documents into a matrix of weighted TF-IDF features.
- **Role in Lab:** Automatically computes TF-IDF representations, down-weighting corpus-wide frequent terms.

NLP Lab 3: Word Embeddings & Foundational Classifiers

Module 1: Continuous Word Embeddings (Word2Vec Skip-Gram)

1.1 Theoretical Foundations

From Discrete to Continuous Representations

Traditional representations (such as One-Hot Vectors or Bag of Words) treat words as discrete, orthogonal entities. In a One-Hot encoding scheme of vocabulary size $|V|$, every word is represented as a high-dimensional vector with a single 1 and $|V| - 1$ zeros:

$$v_{\text{cat}} = [1, 0, 0, \dots, 0]^T, \quad v_{\text{dog}} = [0, 1, 0, \dots, 0]^T$$

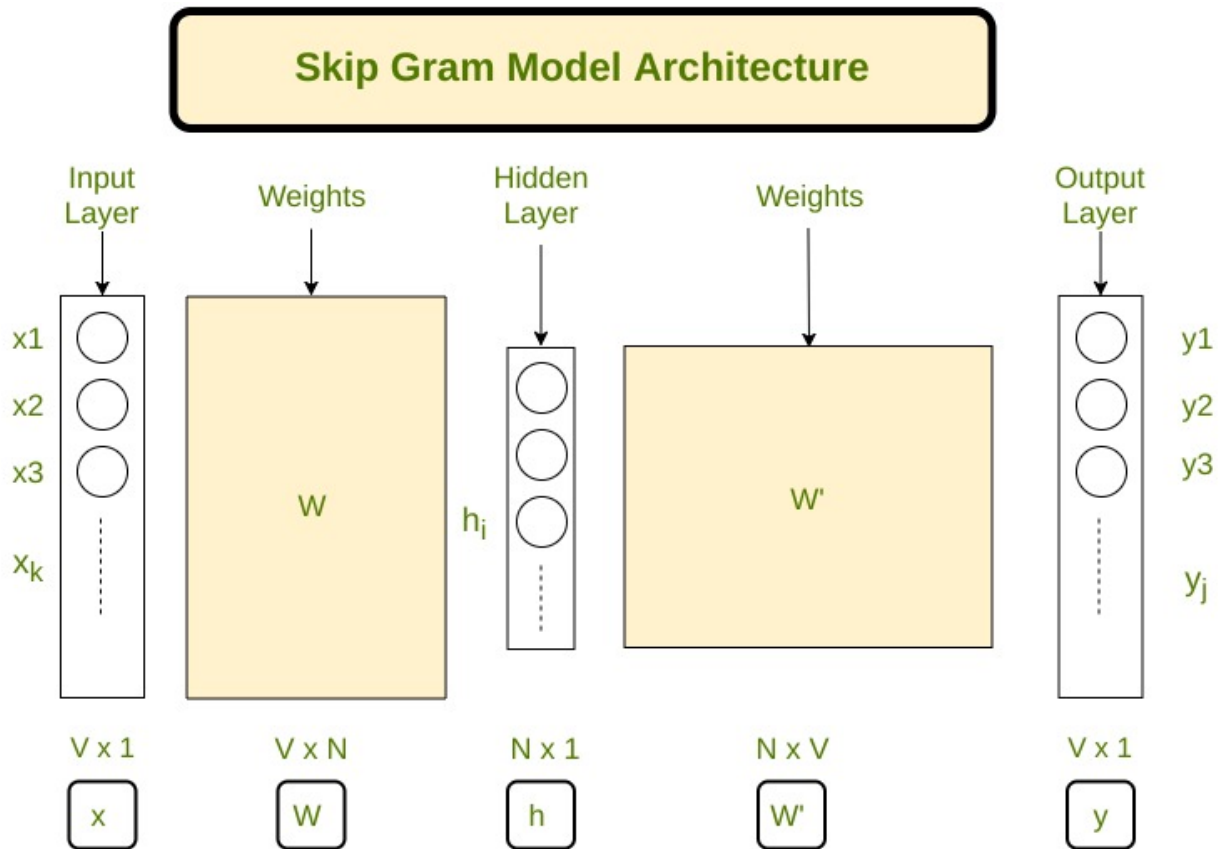
- **Drawback 1 (High Sparsity):** As $|V|$ grows into hundreds of thousands, vectors become computationally inefficient.
- **Drawback 2 (Zero Semantic Similarity):** The dot product $v_{\text{cat}} \cdot v_{\text{dog}} = 0$. The model cannot infer that "cat" and "dog" share semantic context.

Word Embeddings project words into a dense, continuous vector space $\mathbb{R}^d$ (where $d \ll |V|$, typically $d \in [50, 300]$). In this space, words with similar contextual distributions lie close to each other.

The Skip-Gram Architecture

The Skip-gram model learns embeddings by predicting surrounding **context words** w_o given a target **center word** w_c within a sliding window $\mathfrak{c}$:

Maximize $P(w_{t+j} \mid w_t)$ for $-c \leq j \leq c, j \neq 0$



1. Parameters:

- $W_1 \in \mathbb{R}^{|V| \times d}$: Input-to-hidden weight matrix (The **Embedding Matrix** where row i is the vector for word i).
- $W_2 \in \mathbb{R}^{d \times |V|}$: Hidden-to-output weight matrix.

2. Forward Pass Math:

Given a center word index c , its hidden representation is $h = W_{1,c}^\top$. The raw output scores (logits) z for all words in the vocabulary are calculated as:

$$z = W_2^\top h$$

Applying the **Softmax** function yields the predicted probability distribution over the vocabulary:

$$\hat{y}_i = P(w_i | w_c) = \frac{\exp(z_i)}{\sum_{j=1}^{|V|} \exp(z_j)}$$

3. Loss Function (Cross-Entropy):

For a true context word index o , the loss L is:

$$L = -\log P(w_o | w_c) = -z_o + \log \sum_{j=1}^{|V|} \exp(z_j)$$

4. Gradients and Optimization:

The gradient with respect to output score z_i evaluates to:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i \implies e = \hat{y} - y$$

Where $\mathbf{e}$ is the one-hot target vector for the context word. Weights are updated via Stochastic Gradient Descent (SGD):

$$W_2 := W_2 - \alpha(h \cdot e^\top), \quad W_1[c] := W_1[c] - \alpha(W_2 \cdot e)^\top$$

1.2 Code Walkthrough & Implementation Details

1. Optimization Shortcut (Matrix Lookup vs. Matrix Multiplication)

In theory, the hidden layer computes $h = W_1^\top x$. Because x is a one-hot vector with a 1 at index c , multiplying $W_1^\top x$ yields row c of W_1 .

- **In Code:** We skip $W_1^\top x$ matrix multiplication and directly index `W1[center_id]`.
- Similarly, for the error vector $e = \hat{y} - y$, we compute `y_pred` and subtract `1.0` directly from `y_pred[context_id]`.

2. Softmax Numerical Stability

When computing $\exp(z_i)$, large values of z_i can cause numeric overflow (NaN).

- **Fix:** Subtract $\max(z)$ from all elements of z before exponentiating:

$$\text{softmax}(z)_i = \frac{\exp(z_i - \max(z))}{\sum_j \exp(z_j - \max(z))}$$

Module 2: Generative Text Classification (Naive Bayes)

2.1 Theoretical Foundations

Bayes' Theorem for Classification

Naive Bayes is a **generative model**. Given a document $D = (w_1, w_2, \dots, w_n)$ and a set of classes C , it calculates the posterior probability $P(c \mid D)$ for each class $c \in C$:

$$P(c \mid D) = \frac{P(D \mid c)P(c)}{P(D)}$$

Since $P(D)$ is constant across all classes during prediction, we maximize the numerator:

$$\hat{c} = \arg \max_{c \in C} P(c)P(w_1, w_2, \dots, w_n \mid c)$$

The "Naive" Independence Assumption

Naive Bayes assumes that the occurrences of words are conditionally independent given the class label c :

$$P(w_1, w_2, \dots, w_n \mid c) = \prod_{i=1}^n P(w_i \mid c)$$

Thus, the classification rule becomes:

$$\hat{c} = \arg \max_{c \in C} \left[P(c) \prod_{i=1}^n P(w_i \mid c) \right]$$

Parameter Estimation & Laplace Smoothing

1. Prior Probability:

$$P(c) = \frac{N_c}{N_{\text{total}}}$$

(where N_c is the number of documents in class c)

2. Likelihood with Laplace (Add-1) Smoothing:

If a word w_i never appears in class c during training, $P(w_i \mid c) = 0$, causing the entire product to become zero. We apply Laplace smoothing:

$$P(w_i \mid c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$$

Log-Space Computation

Multiplying many probabilities $P(w_i \mid c) < 1$ causes **floating-point underflow**. We transform calculations into log-space (converting multiplication to addition):

$$\hat{c} = \arg \max_{c \in C} \left[\log P(c) + \sum_{i=1}^n \log P(w_i \mid c) \right]$$

2.2 Code Walkthrough & Implementation Details

• Training Phase:

1. Iterate through documents to collect global term counts per class
`word_counts[label][word]`.
2. Compute log priors: `log_priors[c] = np.log(classes[c] / total_docs)`.
3. Precompute log likelihoods for every word in the vocabulary using the smoothed denominator `total_words_in_c + V_size`.

• Prediction Phase:

1. Initialize total score for class c with `log_priors[c]`.
2. Iterate through test words, adding `log_likelihoods[c][word]`. Unseen vocabulary words are skipped.
3. Select the class with the maximum score using `max(scores, key=scores.get)`.

Module 3: Discriminative Text Classification (Logistic Regression)

3.1 Theoretical Foundations

Generative vs. Discriminative

Unlike Naive Bayes (which models the joint distribution $P(D, c)$), Logistic Regression is a **discriminative model**. It models the conditional probability $P(c | D)$ directly by learning decision boundaries.

Model Formulation

For binary classification ($y \in \{0, 1\}$), given a document feature vector $x \in \mathbb{R}^n$:

1. **Linear Combination:**

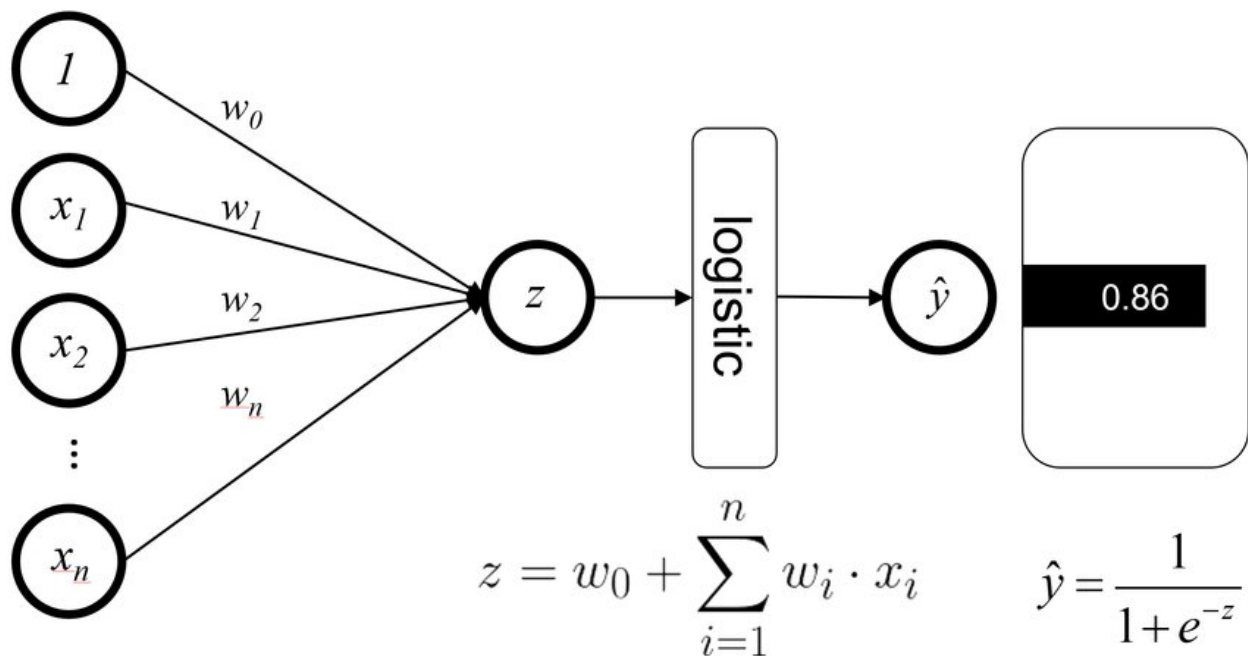
$$z = W^\top x + b = \sum_{j=1}^n w_j x_j + b$$

2. **Sigmoid Activation Function:**

To map $z \in (-\infty, \infty)$ to a valid probability $\hat{y} \in (0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = P(y = 1 | x) = \sigma(W^\top x + b)$$



Binary Cross-Entropy Loss

For a dataset of m samples, the objective function (Log Loss) is:

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Gradient Descent Updates

Partial derivatives with respect to parameters:

$$\frac{\partial J}{\partial W} = \frac{1}{m} X^\top (\hat{y} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

Gradient updates applied iteratively with learning rate α :

$$W := W - \alpha \frac{\partial J}{\partial W}, \quad b := b - \alpha \frac{\partial J}{\partial b}$$

3.2 Code Walkthrough & Implementation Details

- **Vectorization:** Features X are structured as an $m \times n$ matrix (m documents, n vocabulary size).
- **Linear Projection:** Computed in NumPy as `z = np.dot(X, weights) + bias`.
- **Gradient Calculation:**
- Python

```
dw = (1 / m) * np.dot(X.T, (y_pred - y))
```

```
db = (1 / m) * np.sum(y_pred - y)
```

-
- This vectorized update processes all training instances simultaneously.

Module 4: Text Vector Aggregation & Sequence Limitations

4.1 Theoretical Foundations

Mean Embeddings vs. TF-IDF Weighted Embeddings

Logistic Regression requires a fixed-size vector per document. To convert word-level embeddings into a document vector X_{doc} , two common strategies are used:

1. **Unweighted Mean Embedding ("Neural Bag of Words"):**

$$X_{\text{doc}} = \frac{1}{N} \sum_{i=1}^N v_{w_i}$$

2. **TF-IDF Weighted Average Embedding:**

$$X_{\text{doc}} = \frac{\sum_{i=1}^N (\text{TF-IDF}(w_i) \cdot v_{w_i})}{\sum_{i=1}^N \text{TF-IDF}(w_i)}$$

- **Advantage of TF-IDF Weighting:** Reduces the impact of high-frequency, non-informative words (e.g., "is", "the"), giving more weight to salient terms (e.g., "terrible", "excellent").

4.2 The "Bag of Words" Structural Limitation

A fundamental limitation of **any** vector aggregation scheme (Mean or TF-IDF weighted) is that it lacks sensitivity to word sequence order.

Mathematical Proof of the Anagram Dilemma

Consider the two sentences:

- S_1 : "the dog bit the man"
- S_2 : "the man bit the dog"

Let $V = \{\text{"the"}, \text{"dog"}, \text{"bit"}, \text{"man"}\}$.

1. **Term Frequencies (TF):**

- $\text{TF}(S_1) = \{\text{"the"} : 2/5, \text{"dog"} : 1/5, \text{"bit"} : 1/5, \text{"man"} : 1/5\}$
- $\text{TF}(S_2) = \{\text{"the"} : 2/5, \text{"dog"} : 1/5, \text{"bit"} : 1/5, \text{"man"} : 1/5\}$

2. **Inverse Document Frequency (IDF):**

Since IDF is computed over the entire corpus D , $\text{IDF}(w)$ is identical for both sentences.

3. **Aggregation Equivalence:**

Because vector addition is **commutative** ($A + B = B + A$):

$$X_{\text{doc}}(S_1) = \sum_{w \in S_1} \text{Weight}(w) \cdot v_w \equiv \sum_{w \in S_2} \text{Weight}(w) \cdot v_w = X_{\text{doc}}(S_2)$$

Core Takeaway: Because $X_{\text{doc}}(S_1) = X_{\text{doc}}(S_2)$, any downstream linear model (such as Logistic Regression or SVM) will produce identical probabilities for both inputs, despite their opposing meanings. To preserve word order, models must process sequences recurrently (e.g., RNNs, LSTMs) or via position-encoded attention mechanisms (e.g., Transformers).

Summary Comparison Matrix

Property	Naive Bayes	Logistic Regression	Word2Vec (Skip-gram)
Model Family	Generative	Discriminative	Unsupervised Self-Supervised

Optimization Target	Joint Likelihood $P(D, c)$	Conditional Likelihood $P(c \mid D)$	Neighborhood Likelihood $P(w_o \mid w_c)$
Primary Input	Sparse Word Counts	Sparse BoW or Dense Embeddings	One-Hot Word Index
Computational Speed	Extremely Fast (Closed-form counts)	Moderate (Iterative Gradient Descent)	High Compute Cost (Iterative over text corpus)
Handling Unseen Words	Handled via Laplace Smoothing	Handled via zero features or zero vectors	Handled via OOV vector strategies
Preserves Word Order?	No	No (when using BoW/Averaging)	Partially (within context window τ)

NLP Lab 4 Reference Manual: PyTorch Sequence Models & Code Walkthrough

Part 1: PyTorch Essentials for Sequence Processing

1. Core Tensors & Device Management

PyTorch operates on **Tensors**, which are multidimensional arrays optimized for GPU acceleration and automatic differentiation.

```
# Check hardware availability and create a dynamic device target
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Moving tensors to hardware device
x = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.long).to(device)
```

Key Rule for Beginners: Every tensor passed into a PyTorch model (including model parameters themselves) **must reside on the same device**. Mismatched devices (**cpu** vs **cuda:0**) will trigger runtime errors.

2. Structuring Neural Networks (**nn.Module**)

Every custom model in PyTorch inherits from **nn.Module**. You must define:

1. **__init__()**: Instantiates trainable layers (e.g., embeddings, LSTMs, linear layers).
2. **forward()**: Defines the computational graph execution path.

```
class SimpleClassifier(nn.Module):

    def __init__(self, vocab_size, hidden_dim):

        super().__init__()

        self.embedding = nn.Embedding(vocab_size, 300)

        self.fc = nn.Linear(300, hidden_dim)

    def forward(self, x):

        # x shape: (batch_size, seq_len)

        embedded = self.embedding(x)  # shape: (batch_size, seq_len,
300)
```

```

        out = self.fc(embedded)          # shape: (batch_size, seq_len,
hidden_dim)

    return out

```

3. The PyTorch Optimization Pipeline

Training loop execution in PyTorch requires explicit management of gradients:

```

optimizer = optim.Adam(model.parameters(), lr=0.001)

criterion = nn.CrossEntropyLoss()

# Training iteration pattern

optimizer.zero_grad()    # 1. Clear residual gradients from prior
iteration

outputs = model(inputs)  # 2. Perform forward pass

loss = criterion(outputs, targets) # 3. Compute loss value

loss.backward()          # 4. Compute partial derivatives via
backpropagation

optimizer.step()         # 5. Update weights using computed gradients

```

Built-in PyTorch Functionalities Used in Lab 4

1. Pretrained Embeddings (**nn.Embedding.from_pretrained**)

Loads external vectors (like Google News Word2Vec or FastText) into a PyTorch embedding layer.

```

self.embedding = nn.Embedding.from_pretrained(

    embed_matrix,    # Pretrained matrix tensor (vocab_size, embed_dim)

    freeze=False,    # Set to False to allow fine-tuning during
training

    padding_idx=0    # Forces gradient updates for index 0 (<PAD>) to
remain zero

)

```

2. Recurrent Layers (**nn.RNN** and **nn.LSTM**)

PyTorch recurrent units process 3D sequences. Always set `batch_first=True` for intuitive tensor indexing: (`batch_size`, `sequence_length`, `feature_dimension`).

- **nn.RNN Output:**
 - **output:** Sequence of hidden states across all time steps $\rightarrow$ (`batch`, `seq_len`, `hidden_dim * num_directions`).
 - **h_n:** Final hidden state for each element in the batch $\rightarrow$ (`num_layers * num_directions`, `batch`, `hidden_dim`).
- **nn.LSTM Output:**
 - **output:** Sequence of outputs for all steps.
 - (**h_n**, **c_n**): Tuple containing final **hidden states** (h_n) and **cell states** (c_n).
 -

```
# Bidirectional 2-layer LSTM definition

self.lstm = nn.LSTM(

    input_size=300,

    hidden_size=128,

    num_layers=2,

    bidirectional=True,

    batch_first=True

)
```

3. Loss Masking (**ignore_index**)

When working with variable-length sequences, padded locations (<PAD> tokens mapped to index 0) must not influence loss calculation or gradient updates.

```
# Ignores loss calculation for target targets with value = 0

criterion = nn.CrossEntropyLoss(ignore_index=0)
```

4. Batch Matrix Multiplication (**torch.bmm**)

Computes batch-wise matrix products without flattening or looping. Used extensively in **Attention Mechanisms**:

Given:

- Query $Q \in \mathbb{B} \times 1 \times h$
- Key $K \in \mathbb{B} \times S \times h$

```
# Transpose Keys to (batch, hidden_dim, seq_len)
```

```
keys_transposed = K.transpose(1, 2)
```

```
# Batch Multiply: (batch, 1, hidden_dim) x (batch, hidden_dim, seq_len) -> (batch, 1, seq_len)
```

```
attention_scores = torch.bmm(Q, keys_transposed)
```

5. Sampling Methods (**torch.topk** & **torch.multinomial**)

Used in **Autoregressive Text Generation** to restrict predictions to reasonable candidate tokens:

```
# Extract top-K logit values and their corresponding indices
```

```
top_k_logits, top_k_indices = torch.topk(logits, k=5, dim=-1)
```

```
# Sample a single token index from a probability distribution tensor
```

```
sampled_index = torch.multinomial(probs_distribution, num_samples=1)
```

Part 2: Code Walkthrough

Topic 0: Pretrained Embedding Integration (Word2Vec to PyTorch)

1. Initializing Special Tokens & Storage

```
# Initialize vocabulary with PAD and UNK tokens
```

```
vocab = {"<PAD>": 0, "<UNK>": 1}
```

```
# Index 0 -> Zeros for PAD; Index 1 -> Random normal vector for UNK

embedding_weights = [np.zeros(embedding_dim),
np.random.randn(embedding_dim)]
```

- **Explanation:** Index 0 is assigned to <PAD> (used for padding shorter sentences) and mapped to a zero vector so it contributes zero magnitude initially. Index 1 is reserved for <UNK> (out-of-vocabulary words) and initialized with a random Gaussian vector.

2. Mapping Corpus Tokens to Gensim Vectors

```
for sentence in corpus:

    for word in sentence:

        if word not in vocab:

            vocab[word] = len(vocab)

            if word in w2v_model:

                embedding_weights.append(w2v_model[word])

            else:

embedding_weights.append(np.random.randn(embedding_dim))
```

- **Explanation:** Iterates over every word in the dataset. If the word exists in the pre-trained Google Word2Vec model, its pre-trained 300-dimensional vector is extracted. If the word is missing from Google Word2Vec, a random normal vector is generated.

3. Converting to a PyTorch Tensor

```
embedding_matrix = torch.tensor(np.array(embedding_weights),
dtype=torch.float32)
```

- **Explanation:** Converts the list of NumPy array vectors into a 2D PyTorch FloatTensor of shape (vocab_size, 300) ready to be loaded into neural network layers.

Topic 1: Sequence Classification (Vanilla RNN vs. Stacked BiLSTM)

1. Sequence Encoding & Truncation/Padding

```
def encode(sent, vocab, max_len=7):  
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in sent.split()]  
  
    return tokens[:max_len] + [vocab["<PAD>"]] * max(0, max_len -  
len(tokens))
```

- **Explanation:** Transforms raw text sentences into integer sequences. Truncates long sequences to **max_len=7** or appends **<PAD>** (index 0) to short sequences to enforce uniform batch shapes.

2. Vanilla RNN Classifier Architecture

```
class VanillaRNNClassifier(nn.Module):  
  
    def __init__(self, embed_matrix, hidden_dim):  
        super().__init__()  
  
        self.embedding = nn.Embedding.from_pretrained(embed_matrix,  
freeze=False, padding_idx=0)  
  
        self.rnn = nn.RNN(embed_matrix.shape[1], hidden_dim,  
batch_first=True)  
  
        self.fc = nn.Linear(hidden_dim, 1)  
  
    def forward(self, x):  
        embedded = self.embedding(x)  
  
        out, h_n = self.rnn(embedded)  
  
        return self.fc(h_n.squeeze(0)).squeeze(1)
```

- **nn.Embedding.from_pretrained(..., freeze=False, padding_idx=0):** Loads the Word2Vec matrix. **freeze=False** enables fine-tuning during training, while **padding_idx=0** prevents index 0 (**<PAD>**) from receiving gradient updates.

- **self.rnn(...)**: Processes embedded sequences step-by-step. **batch_first=True** expects shapes of (batch_size, sequence_length, embedding_dim).
- **h_n.squeeze(0)**: Extracts the final hidden state vector across time steps and passes it to a dense linear layer for binary sentiment output.

3. Stacked Bidirectional LSTM Architecture

```
class StackedBiLSTMClassifier(nn.Module):

    def __init__(self, embed_matrix, hidden_dim, num_layers=2):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        self.lstm = nn.LSTM(

            embed_matrix.shape[1], hidden_dim,

            num_layers=num_layers, batch_first=True,
bidirectional=True

        )

        self.fc = nn.Linear(hidden_dim * 2, 1)

    def forward(self, x):

        embedded = self.embedding(x)

        out, (h_n, c_n) = self.lstm(embedded)

        forward_h = h_n[-2, :, :]

        backward_h = h_n[-1, :, :]

        bi_hidden = torch.cat((forward_h, backward_h), dim=1)

        return self.fc(bi_hidden).squeeze(1)
```

- **num_layers=2, bidirectional=True**: Stacked 2-layer LSTM where processing occurs both forward (left-to-right) and backward (right-to-left).
- **h_n[-2, :, :]** and **h_n[-1, :, :]**: Retrieves the final hidden states from the top layer for both forward (-2) and backward (-1) passes.

- **torch.cat(..., dim=1)**: Concatenates both directions into a single representation vector of size `hidden_dim * 2` before classification.

4. Binary Classification Loss & Optimizer Setup

```
criterion = nn.BCEWithLogitsLoss()
```

```
opt_rnn = optim.Adam(rnn_model.parameters(), lr=0.01)
```

- **nn.BCEWithLogitsLoss()**: Combines a Sigmoid layer and Binary Cross-Entropy loss in a single layer for numerical stability.

Topic 2: Sequence Labeling with Bidirectional RNN

1. Bidirectional POS Tagging Network

```
class BiRNNSequenceLabeler(nn.Module):

    def __init__(self, embed_matrix, hidden_dim, tagset_size):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        self.rnn = nn.RNN(embed_matrix.shape[1], hidden_dim,
batch_first=True, bidirectional=True)

        self.fc = nn.Linear(hidden_dim * 2, tagset_size)

    def forward(self, x):

        embeds = self.embedding(x)

        rnn_out, _ = self.rnn(embeds) # (batch, seq_len, hidden_dim *
2)

        logits = self.fc(rnn_out)      # (batch, seq_len, tagset_size)

        return logits
```

- **Explanation:** Unlike classification (which returns one vector per sequence), sequence labeling outputs predictions for *every token step*. `rnn_out` maintains sequence dimension `seq_len`, and `self.fc` projects each token representation to the POS tag set size.

2. Loss Masking with `ignore_index`

```
pos_criterion = nn.CrossEntropyLoss(ignore_index=0)
```

```
# Inside Training Loop:
```

```
logits = pos_model(X_pos)
```

```
loss = pos_criterion(logits.view(-1, len(tag_to_ix)), y_pos.view(-1))
```

- **`logits.view(-1, tagset_size)`:** Flattens 3D output tensors (`batch`, `seq_len`, `tagset_size`) into 2D tensors to match standard Cross-Entropy requirements.
- **`ignore_index=0`:** Automatically ignores loss computation and gradient backpropagation for `<PAD>` tokens.

Topic 3: Language Modeling & Sentence Generation with Stacked LSTM

1. Data Structuring via Sliding Window

```
seq_length = 5
```

```
lm_X, lm_Y = [], []
```

```
encoded_words = [vocab.get(w, vocab["<UNK>"]) for w in words]
```

```
for i in range(0, len(encoded_words) - seq_length):
```

```
    lm_X.append(encoded_words[i : i + seq_length])
```

```
    lm_Y.append(encoded_words[i + 1 : i + seq_length + 1])
```

- **Explanation:** Constructs an autoregressive sequence dataset. For every sequence slice of length 5 (`lm_X`), the target label sequence (`lm_Y`) is shifted forward by exactly 1 token position.

2. Stacked LSTM Language Model

```
class StackedLSTMLanguageModel(nn.Module):

    def __init__(self, embed_matrix, hidden_dim=128, num_layers=2):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        embed_dim = embed_matrix.shape[1]

        self.lstm = nn.LSTM(embed_dim, hidden_dim,
num_layers=num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_dim, embed_matrix.shape[0])

    def forward(self, x, hidden=None):

        embeds = self.embedding(x)

        out, hidden = self.lstm(embeds, hidden)

        logits = self.fc(out)

        return logits, hidden
```

- **Explanation:** Takes input sequences and outputs predictions over the entire vocabulary (`embed_matrix.shape[0]`) for each token position. Passes hidden state tuples across time steps during evaluation/generation.

3. Autoregressive Generation with Temperature Sampling

```
with torch.no_grad():

    for _ in range(generate_length):

        logits, hidden = model(input_eval, hidden)

        # Scale logits using temperature
```

```

last_logits = logits[0, -1, :] / temperature

probs = torch.softmax(last_logits, dim=-1)

# Sample index based on probability distribution

pred_idx = torch.multinomial(probs, num_samples=1).item()

# Prepare input for next step

input_eval = torch.tensor([[pred_idx]],
dtype=torch.long).to(device)

generated_words.append(idx2word.get(pred_idx, "<UNK>"))

```

- **logits[0, -1, :] / temperature:** Extracts predictions for the last predicted word position and divides by **temperature** (lower values make distribution sharper/more deterministic; higher values increase diversity).
- **torch.multinomial(probs, num_samples=1):** Samples the next token randomly according to the Softmax probability distribution.

Topic 4: Encoder-Decoder Seq2Seq Machine Translation

1. Building English Word2Vec Matrix for Encoder

```

en_embedding_matrix = torch.zeros((len(en_vocab), 300))

for word, idx in en_vocab.items():

    if word in w2v_google:

        en_embedding_matrix[idx] = torch.tensor(w2v_google[word])

    else:

        if word != "<PAD>":

            en_embedding_matrix[idx] = torch.randn(300) * 0.1

```

- **Explanation:** Creates a specialized embedding tensor for the source language (English) using pre-trained Word2Vec vectors. Unseen English tokens are randomly initialized.

2. Encoder Architecture

```
class Seq2SeqEncoder(nn.Module):

    def __init__(self, embed_matrix, hidden_dim):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        self.lstm = nn.LSTM(embed_matrix.shape[1], hidden_dim,
batch_first=True)

    def forward(self, x):

        embedded = self.embedding(x)

        _, (hidden, cell) = self.lstm(embedded)

        return hidden, cell
```

- **Explanation:** Processes the source English text. Discards sequence outputs and returns only final hidden (**hidden**) and cell (**cell**) states, which act as context vectors containing sentence information.

3. Decoder Architecture

```
class Seq2SeqDecoder(nn.Module):

    def __init__(self, vocab_size, embed_dim, hidden_dim):

        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)

        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)

        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden, cell):
```

```

embedded = self.embedding(x)

out, (hidden, cell) = self.lstm(embedded, (hidden, cell))

preds = self.fc(out)

return preds, hidden, cell

```

- **Explanation:** Processes destination Bengali tokens step-by-step. Receives initial context state vectors (**hidden, cell**) from the Encoder and updates them sequentially as translated words are generated.

4. Seq2Seq Parent Model with Teacher Forcing

```

class Seq2SeqTranslation(nn.Module):

    def forward(self, src, trg):

        batch_size = src.shape[0]

        trg_len = trg.shape[1]

        trg_vocab_size = self.decoder.fc.out_features

        outputs = torch.zeros(batch_size, trg_len,
                               trg_vocab_size).to(device)

        # 1. Pass English sentence through encoder
        hidden, cell = self.encoder(src)

        # 2. First decoder input is <SOS> token
        decoder_input = trg[:, 0].unsqueeze(1)

        # 3. Teacher forcing loop
        for t in range(1, trg_len):

```

```

        output, hidden, cell = self.decoder(decoder_input, hidden,
cell)

        outputs[:, t, :] = output.squeeze(1)

        decoder_input = trg[:, t].unsqueeze(1) # Feed actual
ground-truth token

    return outputs

```

- **Teacher Forcing Mechanism:** During training, instead of feeding the model's own (potentially incorrect) previous prediction, `decoder_input = trg[:, t].unsqueeze(1)` feeds the actual target Bengali word from the ground-truth data at step `t` to stabilize training convergence.

5. Translation Inference Loop

```

with torch.no_grad():

    hidden, cell = model.encoder(src_tensor)

    decoder_input = torch.tensor([[bn_vocab["<SOS>"]]],
dtype=torch.long).to(device)

    translated_words = []

    for _ in range(max_bn_len):

        output, hidden, cell = model.decoder(decoder_input, hidden,
cell)

        pred_idx = output.argmax(dim=-1).item()

        if pred_idx == bn_vocab["<EOS>"]:

            break

    translated_words.append(inv_bn_vocab.get(pred_idx, ""))

```

```
decoder_input = torch.tensor([[pred_idx]],
dtype=torch.long).to(device)
```

- **Explanation:** During translation testing/inference, teacher forcing is disabled. The model feeds its own argmax prediction (`pred_idx`) as the input token to the next step until `<EOS>` (end of sentence) is generated.

Task / Topic	Model Class	Inputs / Shapes	Key Operations & PyTorch Functions
Topic 0: Pretrained Vectors	<code>build_pretrained_embedding_matrix()</code>	Corpus tokens → Tensor	<code>api.load()</code> , <code>torch.tensor(..., dtype=torch.float32)</code>
Topic 1: Classification	<code>VanillaRNNClassifier</code>	(B, T) token IDs	<code>nn.RNN</code> , <code>h_n.squeeze(0)</code> , <code>nn.BCEWithLogitsLoss()</code>
Topic 1: Classification	<code>StackedBiLSTMClassifier</code>	(B, T) token IDs	<code>nn.LSTM(num_layers=2, bidirectional=True)</code> , <code>torch.cat((h_fwd, h_bwd), dim=1)</code>
Topic 2: Tagging	<code>BiRNNSequenceLabeler</code>	(B, T) token IDs	<code>nn.RNN(bidirectional=True)</code> , <code>logits.view(-1, C)</code> , <code>CrossEntropyLoss(ignore_index=0)</code>
Topic 3: LM Generation	<code>StackedLSTMLanguageModel</code>	(B, T) sliding window	<code>nn.LSTM(num_layers=2)</code> , <code>torch.multinomial(probs, 1)</code>
Topic 4: Translation	<code>Seq2SeqTranslation</code>	Parallel src/trg Tensors	Teacher Forcing iteration, <code>Encoder(h, c)</code> context passing

NLP Lab 5: Transformer Code Walk-through

Block 1: Environment & Seed Setup

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import math
import random
import itertools

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
torch.manual_seed(42)
random.seed(42)
```

- `import ...`: Loads PyTorch neural network modules (`nn`), optimization algorithms (`optim`), and data handling utilities (`DataLoader`, `TensorDataset`).
- `device = ...`: Automatically detects if a GPU (`cuda`) is available to accelerate matrix multiplications; otherwise, defaults to `cpu`.
- `torch.manual_seed(42)`: Locks the random number generator so weight initialization and data shuffling yield the exact same results every time you run the script.

Block 2: Toy Corpus Generation

```
pos_adjs = ["amazing", "brilliant", "fantastic", "great", "wonderful", "superb"]
pos_nouns = ["movie", "film", "story", "acting", "cinema", "masterpiece"]
pos_endings = ["loved it", "highly recommend", "a joy to watch", "perfect", "enjoyed every minute"]

# ... (negative lists omitted for brevity) ...

pos_sentences = [f"{a} {n} {e}" for a, n, e in itertools.product(pos_adjs, pos_nouns, pos_endings)]
pos_sentences = random.sample(pos_sentences, 600)

sentences = pos_sentences + neg_sentences
labels = [1] * 600 + [0] * 600 # 1: Positive, 0: Negative

combined = list(zip(sentences, labels))
```

```
random.shuffle(combined)
sentences, labels = zip(*combined)
```

- `itertools.product(...)`: Generates every possible combination of adjective + noun + ending to create thousands of synthetic reviews.
- `random.sample(..., 600)`: Randomly selects exactly 600 positive and 600 negative reviews to create a perfectly balanced dataset of 1,200 samples.
- `labels = ...`: Creates a corresponding list of integer targets (1 for positive, 0 for negative).
- `random.shuffle(...)`: Mixes the dataset so the model doesn't see all positive reviews first and all negative reviews second.

Block 3: Vocabulary and Tokenization

```
vocab = {"<PAD>": 0, "<UNK>": 1}
for sent in sentences:
    for word in sent.split():
        if word not in vocab:
            vocab[word] = len(vocab)

MAX_LEN = 8
def encode(sent):
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in sent.split()]
    return tokens[:MAX_LEN] + [vocab["<PAD>"]] * max(0, MAX_LEN - len(tokens))

X = torch.tensor([encode(s) for s in sentences], dtype=torch.long)
y = torch.tensor(labels, dtype=torch.float32)

dataset = TensorDataset(X, y)
train_loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

- `vocab = ...`: A dictionary mapping words to unique integer IDs. Initializes `<PAD>` for filling short sentences and `<UNK>` for unknown words.
- `encode(sent)`: Converts a text string into a list of integers. Truncates lists longer than `MAX_LEN` (8) and appends `<PAD>` (0) to shorter ones.
- `torch.tensor(...)`: Converts Python lists into PyTorch tensors. Features (X) are long (integers) for embedding lookups; labels (y) are float32 for binary loss calculation.
- `DataLoader(...)`: Wraps the dataset into an iterator that serves 32 samples at a time (`batch_size=32`).

Block 4: Positional Encoding Module

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return x + self.pe[:, :x.size(1), :]
```

- `torch.zeros(...)`: Initializes an empty matrix to hold the positional signals.
- `torch.sin / torch.cos`: Applies alternating sine and cosine waves of different frequencies to generate unique positional signatures for every token index.
- `self.register_buffer('pe', pe)`: Saves the `pe` tensor as part of the model's state so it moves to the GPU, but prevents the optimizer from trying to update it.
- `x + self.pe[...]`: In the forward pass, adds the positional wave values directly to the token embeddings.

Block 5: Transformer Classifier Architecture

```
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4, num_layers=2, num_classes=1, max_len=32):
        super().__init__()
        self.d_model = d_model

        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_encoder = PositionalEncoding(d_model, max_len=max_len)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=256, batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.classifier = nn.Linear(d_model, num_classes)

    def forward(self, x, padding_mask=None):
```

```

x = self.embedding(x) * math.sqrt(self.d_model)
x = self.pos_encoder(x)

encoded = self.transformer_encoder(x, src_key_padding_mask=padding_mask)
pooled = encoded.mean(dim=1)
logits = self.classifier(pooled).squeeze(-1)
return logits

```

- `nn.Embedding`: A lookup table converting integer token IDs into dense vectors of size `d_model`. `padding_idx=0` ensures the <PAD> token always outputs zeros.
- `nn.TransformerEncoderLayer`: A single Transformer block containing Multi-Head Self-Attention and a FeedForward network. `batch_first=True` expects inputs as (Batch, Sequence, Features).
- `nn.TransformerEncoder`: Stacks multiple `encoder_layer` blocks sequentially.
- `x * math.sqrt(self.d_model)`: Scales embeddings up before adding positional encodings.
- `encoded.mean(dim=1)`: Mean Pooling. Averages the contextualized token vectors across the sequence length to compress the sentence into a single 1D vector.
- `squeeze(-1)`: Removes the trailing dimension from the classifier output to match the 1D target label shape.

Block 6: Training Loop

```

model = TransformerClassifier(vocab_size=len(vocab), d_model=64, nhead=2, num_layers=2,
                             max_len=MAX_LEN).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.BCEWithLogitsLoss()

model.train()
for epoch in range(5):
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        padding_mask = (inputs == 0).to(device)

        optimizer.zero_grad()
        logits = model(inputs, padding_mask=padding_mask)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()

```

- `optim.Adam`: The optimization algorithm that updates network weights based on computed gradients.

- `BCEWithLogitsLoss`: Binary Cross Entropy loss. It automatically applies a Sigmoid activation to the raw output logits.
- `padding_mask = (inputs == 0)`: Creates a boolean tensor where True marks a <PAD> token. Passed to the Transformer so self-attention ignores padding words.
- `optimizer.zero_grad()`: Clears leftover gradients from the previous batch to prevent accumulation.
- `loss.backward()`: Calculates the derivative (gradient) of the loss with respect to every weight in the network.
- `optimizer.step()`: Adjusts the weights in the direction that minimizes the loss.

Block 7: Inference Function

```
def predict(text, model, vocab, max_len=MAX_LEN):
    model.eval()
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in text.split()]
    padded = tokens[:max_len] + [vocab["<PAD>"]] * max(0, max_len - len(tokens))

    input_tensor = torch.tensor([padded], dtype=torch.long).to(device)
    pad_mask = (input_tensor == 0).to(device)

    with torch.no_grad():
        logit = model(input_tensor, padding_mask=pad_mask)
        prob = torch.sigmoid(logit).item()
```

- `model.eval()`: Switches the model to evaluation mode, disabling training-specific behaviors like Dropout.
- `with torch.no_grad()`: Suspends PyTorch's gradient tracking engine, saving significant memory and compute time during inference.
- `torch.sigmoid(logit).item()`: Converts the raw logit score into a probability between 0.0 and 1.0, extracting it from the PyTorch tensor as a standard Python float.

Transformer Tensor Shapes & Data Flow

Understanding the dimensional transformations of data as it passes through the Encoder-Only Transformer. The shapes assume the following hyperparameters based on our toy corpus training:

- `B` = Batch Size (e.g., 32)

- T = Sequence Length / MAX_LEN (e.g., 8)
- D = Embedding Dimension / d_{model} (e.g., 64)
- C = Number of Classes (1 for Binary Classification)

Shape Transformation Summary

Processing Stage	Tensor Shape	Description
1. Raw Input Tokens	$(B, T) \rightarrow (32, 8)$	Integer indices mapped from the vocabulary.
2. nn.Embedding	$(B, T, D) \rightarrow (32, 8, 64)$	Each integer is mapped to a dense vector of size D .
3. Positional Encoding	$(B, T, D) \rightarrow (32, 8, 64)$	Sine/Cosine wave values are added; shape remains identical.
4. Transformer Encoder	$(B, T, D) \rightarrow (32, 8, 64)$	Self-attention processes context. Input and output shapes match exactly.
5. Mean Pooling	$(B, D) \rightarrow (32, 64)$	Averages across the Sequence dimension (T) to create a single vector per sentence.
6. Linear Classifier	$(B, C) \rightarrow (32, 1)$	Projects the D -dimensional feature vector down to a single logit per sentence.
7. Squeeze	$(B,) \rightarrow (32,)$	Removes the trailing dimension to match the 1D target label tensor.

Detailed Stage-by-Stage Breakdown

1. Input Data & Masking

- Input Tensor: $x = \text{inputs}$

Shape: $(\text{Batch_Size}, \text{Max_Length}) \rightarrow (32, 8)$

- Padding Mask: $\text{padding_mask} = (\text{inputs} == 0)$

Shape: $(\text{Batch_Size}, \text{Max_Length}) \rightarrow (32, 8)$

Type: Boolean matrix used to hide <PAD> tokens from the self-attention mechanism.

2. Embedding & Positional Injection

- Operation: $x = \text{self.embedding}(x) * \text{math.sqrt}(\text{self.d_model})$

Shape: (32, 8) becomes (32, 8, 64)

- Operation: `x = self.pos_encoder(x)`

Shape: Remains (32, 8, 64). Positional encoding matrix of shape (1, 8, 64) is broadcast-added to the embeddings.

3. Transformer Encoder Block

- Operation: `encoded = self.transformer_encoder(x, src_key_padding_mask=padding_mask)`

Shape: Remains (32, 8, 64)

Context: Inside the encoder, Multi-Head Attention projects this into Queries, Keys, and Values of shape (32, 8, 64), splits them into 2 heads of shape (32, 2, 8, 32), computes attention, and concatenates back to (32, 8, 64).

4. Mean Pooling

- Operation: `pooled = encoded.mean(dim=1)`

Shape: (32, 8, 64) is collapsed to (32, 64)

Context: We must reduce the time sequence (8 tokens) into a single aggregate representation so the Linear layer can classify the sentence as a whole.

5. Classification Head & Loss

- Operation: `logits = self.classifier(pooled)`

Shape: (32, 64) becomes (32, 1)

- Operation: `logits = logits.squeeze(-1)`

Shape: (32, 1) becomes (32,)

Context: Matches the `y_target` shape (32,) so `BCEWithLogitsLoss` can compare them directly.